OBJECT-ORIENTED PROGRAMMING MINI PROJECT

A PROJECT REPORT ON

INVENTORY MANAGEMENT SYSTEM

A Java-based web application for product catalogue management,
stock movement, financial calculation and transaction reporting

Submitted by

Siddhi Dherange	25
Jayati Hembade	18
Akram Kadri	21

Academic Year 2026

CERTIFICATE

This is to certify that the project report entitled

“Inventory Management System”

This project has been developed and submitted as a mini project for the Object-Oriented Programming subject. The project demonstrates the application of Java classes, objects, encapsulation, collections, HTTP communication and web application development to solve an inventory management problem.

The work has been carried out by the following students:

Siddhi Dherange 25

Jayati Hembade 18

Akram Kadri 21

Project Guide	Subject Teacher	Head of Department

DECLARATION

We hereby declare that the project entitled **“Inventory Management System”** is our original work developed for the Object-Oriented Programming mini project. The system has been implemented using Java as the primary programming language, with a browser-based frontend and Docker-based deployment configuration.

All design, implementation, testing and documentation work described in this report relates to the project developed by our group. External libraries, platform documentation and development tools used during the work are acknowledged in the references section.

Siddhi Dherange	Jayati Hembade	Akram Kadri
Roll No. 25	Roll No. 18	Roll No. 21

ACKNOWLEDGEMENT

We express our sincere gratitude to our subject teacher and department for providing us with the opportunity to develop this Object-Oriented Programming mini project. The project helped us understand how Java concepts can be applied to a practical business problem.

We are thankful for the guidance and academic support received throughout the planning, implementation, testing and deployment stages. We also acknowledge the use of Java development tools, browser technologies, Docker and Render for building and presenting the system.

Finally, we thank all group members for their cooperation in designing the interface, implementing the Java backend, testing inventory operations and preparing the final project demonstration.

ABSTRACT

Inventory Management System is a web-based application developed to manage products and warehouse stock through a simple interactive dashboard. The application implements the operations specified in the mini project problem statement: listing products, displaying product information, purchasing or restocking, shipping or selling, balancing stock, calculating inventory value and profit, and generating purchase and transaction reports for a selected date range.

The main application logic is implemented in Java using object-oriented programming. The system contains separate classes for products, transactions, inventory operations and HTTP server responsibilities. A Java HTTP server exposes JSON endpoints that are consumed by a responsive HTML, CSS and JavaScript frontend. The application validates quantities, prevents shipping beyond available stock, updates stock after every movement and records transactions for reporting.

The project is packaged for deployment using Docker. A multi-stage Dockerfile compiles the Java source with a Java Development Kit and runs the compiled application in a Java Runtime Environment. The deployment configuration is compatible with Render and includes a health-check endpoint.

TABLE OF CONTENTS

1. Introduction 1

2. Problem Definition and Objectives 2

3. System Analysis and Requirements 3

4. System Design 5

5. Object-Oriented Implementation 7

6. Functional Modules 10

7. Web Interface and API 14

8. Deployment and Testing 17

9. Results and Screenshots 19

10. Conclusion and Future Scope 21

References 23

1. INTRODUCTION

Inventory is the collection of products maintained by an organization for sale or operational use. Accurate stock information is important because an organization must know what products are available, how many units are present, when stock is moving, and whether the activity is generating profit.

Manual inventory recording can lead to incorrect quantities, delayed updates and difficulty in preparing reports. The Inventory Management System developed in this project provides a digital interface for managing product information and stock movement. It combines a Java object-oriented backend with a web frontend so that inventory operations can be performed through a browser.

The project is based on the Group C mini project requirement for an inventory management system. It has been designed as an educational application, with the main focus on Java OOP concepts, clear business rules and interactive presentation.

1.1 Project Scope

- Maintain a product catalogue with product IDs, categories, stock and prices.
- Record products purchased from suppliers.
- Record products shipped or sold to customers.
- Calculate current stock, inventory value, revenue and profit.
- Identify products with low stock.
- Display transactions within a selected date range.
- Deploy the application as a web service.

1.2 Existing System

In a basic manual system, product information and stock changes may be written in registers or maintained in separate files. Such a system makes it difficult to calculate current stock and profit immediately. It also makes it difficult to produce a date-wise movement report.

1.3 Proposed System

The proposed system centralizes product and transaction information in a Java inventory object. Every purchase increases stock, every shipment decreases stock after validation, and every movement is recorded. The dashboard calculates important values from the current application state and presents them through a browser.

2. PROBLEM DEFINITION AND OBJECTIVES

2.1 Problem Definition

To design and implement an inventory management system using maximum practical features of Object-Oriented Programming in Java. The system should support product listing, individual product information, purchase, shipping, stock balance, loss and profit calculation, and purchase reporting between selected dates.

2.2 Objectives

1. To create a Java-based inventory domain model.
2. To apply encapsulation using classes with private data members.
3. To represent products and inventory movements as objects.
4. To update stock accurately after purchase and shipping operations.
5. To prevent invalid quantities and shipments exceeding available stock.
6. To calculate inventory value, revenue, cost and profit.
7. To generate a date-filtered transaction report.
8. To provide an interactive browser-based interface.
9. To package and deploy the application using Docker and Render.

2.3 Significance

The project connects classroom concepts with a practical business workflow. It demonstrates that OOP is not limited to isolated examples: classes can model products, transactions and business services, while methods can protect and modify the state according to defined rules.

3. SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Functional Requirements

Requirement	Description
Product listing	Display all products with ID, name, category, available stock, selling price and margin.
Product information	Show individual product information in the catalogue cards and product selector.
Purchase	Add purchased quantity to available stock and record the purchase amount at cost price.
Shipping	Subtract shipped quantity only when sufficient stock is available and record sale amount at selling price.
Stock balance	Display total units and current stock for each product.
Financial calculation	Calculate inventory value, revenue, cost of shipped goods and profit.
Report	Display purchase and shipping transactions filtered by start and end dates.
Deployment health	Provide an endpoint that confirms the Java server is running.

3.2 Non-Functional Requirements

- **Usability:** The dashboard should be understandable without command-line interaction.
- **Reliability:** Invalid operations should not modify stock.
- **Maintainability:** Business logic should be separated into meaningful classes and methods.
- **Portability:** The application should run locally and inside a Docker container.
- **Responsiveness:** The interface should adapt to smaller screens.

3.3 Hardware and Software Requirements

Category	Requirement
Development system	Any modern Windows, Linux or macOS computer.
Language	Java 17 or newer.
Frontend	Modern web browser with JavaScript enabled.

Deployment	Docker-compatible cloud service such as Render.
Source control	Git and GitHub.

4. SYSTEM DESIGN

4.1 Architecture

The system follows a simple three-layer arrangement. The presentation layer contains the browser interface. The server layer receives requests and returns JSON. The domain layer contains the Java classes that maintain products, transactions and inventory calculations.

Browser Interface
HTML + CSS + JavaScript

↓ HTTP requests and JSON responses

Java HTTP Server
Request routing and validation

↓ method calls

Inventory Domain Model
Product + Transaction + Inventory

4.2 Main Classes

Class	Responsibility
InventoryServer	Starts the server, seeds products, handles API requests, serves frontend files and returns JSON.
Product	Represents a product and stores its identity, category, stock and prices.
Transaction	Represents a purchase or shipping movement with quantity, amount and date.
Inventory	Owns the product catalogue and transaction history and performs business operations.

4.3 Data Flow

1. The frontend requests product and dashboard data.
2. The server reads the request path and query values.
3. The Inventory object calculates or updates the requested information.
4. The server serializes the result as JSON.
5. The frontend renders the returned values.

5. OBJECT-ORIENTED IMPLEMENTATION

5.1 Product Encapsulation

```
static final class Product {
    private final String id;
    private final String name;
    private final String category;
    private int stock;
    private final double costPrice;
    private final double sellingPrice;
}
```

The Product class encapsulates all information belonging to an inventory item. The fields are private, so the class controls access to its internal state. Product identity and pricing are initialized through the constructor. Stock is mutable because stock changes during purchases and shipping.

5.2 Transaction Object

The Transaction class stores the type of movement, product, quantity, customer or supplier name, total amount and date. Creating a separate transaction object preserves an auditable history rather than only changing the current stock number.

5.3 Inventory Composition

```
private final List<Product> products = new ArrayList<>();
private final List<Transaction> transactions = new ArrayList<>();
```

The Inventory class is composed of Product and Transaction objects. This is an example of object composition: the Inventory object manages collections of related domain objects.

5.4 Validation and Controlled State Change

```
if (quantity <= 0)
    throw new IllegalArgumentException(
        "Quantity must be greater than zero");

if ("shipping".equals(action)
    && product.stock < quantity)
    throw new IllegalArgumentException(
        "Insufficient stock for " + product.name);
```

Validation is performed before stock is changed. Therefore an invalid operation produces an error and leaves the inventory state unchanged.

6. FUNCTIONAL MODULES

6.1 Product Catalogue

The system seeds six sample products: Wireless Headphones, Mechanical Keyboard, USB-C Hub, Smart Desk Lamp, Portable SSD 1TB and Webcam Full HD. Each item has a unique ID, category, stock quantity, cost price and selling price.

6.2 Purchase Module

A purchase or restock increases product stock. The purchase total is calculated as quantity multiplied by cost price. The movement is stored as a Transaction with type `purchase`.

6.3 Shipping Module

A shipping or sale operation decreases product stock. Before the change, the system checks that the available stock is greater than or equal to the requested quantity. The shipping total uses selling price and the movement is stored as type `shipping`.

6.4 Balance Stock Module

The balance module displays the stock for each product and calculates the total units across the catalogue. Products below 25 units are identified as low-stock items.

6.5 Loss and Profit Calculation

Inventory value is calculated from the current stock multiplied by cost price. Revenue is the total of shipping transactions. The cost of goods sold is the quantity shipped multiplied by product cost price. Profit is revenue minus the cost of shipped goods.

6.6 Purchase Report Module

The report module accepts optional start and end dates. It filters transactions whose dates fall within the inclusive range and sorts matching transactions by date before returning them to the interface.

7. WEB INTERFACE AND API

7.1 Dashboard

The dashboard displays product count, units in stock, inventory value and profit as summary cards. The product catalogue provides search support and shows stock and margin values for every product.

7.2 Warehouse Action Form

The movement form lets a user select purchase or shipping, select a product, enter quantity and optionally enter a customer or supplier. After submission, the interface shows a confirmation or error notice and refreshes the current inventory values.

7.3 API Endpoints

Method	Endpoint	Purpose
GET	/api/inventory	Returns dashboard totals.
GET	/api/inventory?view=products	Returns the complete product catalogue.
GET	/api/inventory? view=report&from=DATE&to=DATE	Returns filtered transactions.
POST	/api/inventory	Processes purchase and shipping form data.
GET	/api/health	Returns the server health status for Render.

7.4 JSON and Frontend Synchronization

The Java server returns JSON objects and arrays. JavaScript reads the responses with the Fetch API, updates the DOM and reloads the product list and dashboard after every successful movement. Product and report text is HTML-escaped before being inserted into the page.

8. DEPLOYMENT AND TESTING

8.1 Docker Deployment

The project uses a multi-stage Dockerfile. The build stage uses `eclipse-temurin:17-jdk` to compile Java source files. The runtime stage uses `eclipse-temurin:17-jre`, copies compiled classes and frontend files, and starts the Java server.

```
FROM eclipse-temurin:17-jdk AS build
WORKDIR /app
COPY src ./src
RUN mkdir -p target/classes && \
    javac -d target/classes \
    src/main/java/com/inventory/InventoryServer.java

FROM eclipse-temurin:17-jre
COPY --from=build /app/target/classes ./target/classes
COPY public ./public
CMD ["java", "-cp", "target/classes",
     "com.inventory.InventoryServer"]
```

8.2 Render Configuration

Render is configured to deploy the service as a Docker web service. The server reads the PORT variable provided by Render. No additional secret or environment variable is required for the current in-memory implementation.

8.3 Testing

Test case	Expected result	Status
Compile Java source	Classes are generated successfully.	Passed
List products	Six seeded products are returned.	Passed
Purchase five units	Stock increases and transaction is recorded.	Passed
Ship two units	Stock decreases and transaction is recorded.	Passed
Ship more than available stock	Operation is rejected without changing stock.	Passed
Generate report	Transactions are returned for selected dates.	Passed
Health endpoint	Server returns HTTP 200 and status OK.	Passed
Homepage	Dashboard is served successfully.	Passed

9. RESULTS AND SCREENSHOTS

The deployed application provides a functional inventory dashboard. The first screenshot shows the initial catalogue and stock values. The second screenshot shows a purchase operation after five units have been added to Wireless Headphones, with updated stock and report date controls.

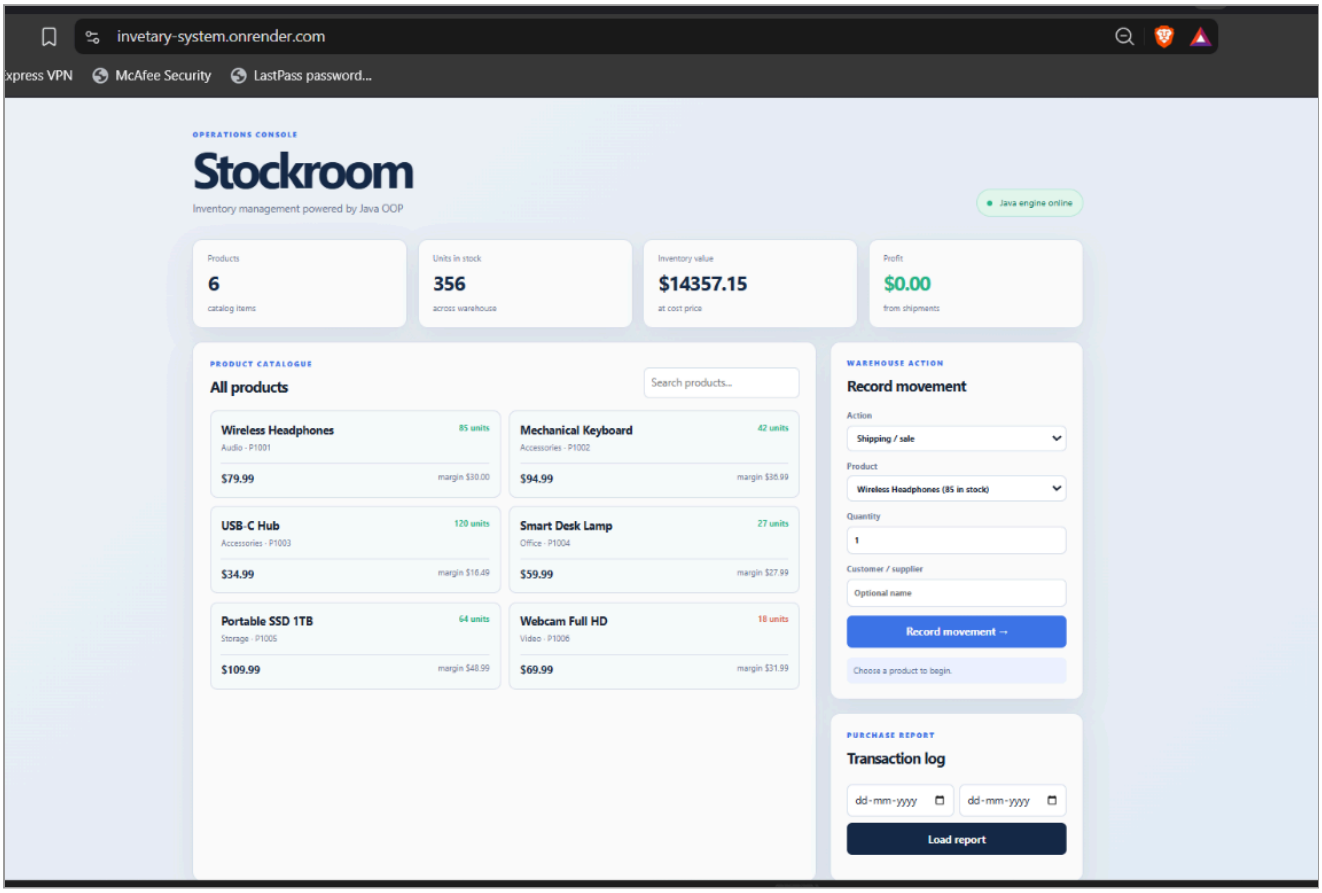


Figure 1: Deployed Stockroom dashboard with catalogue, stock summary and warehouse action controls.

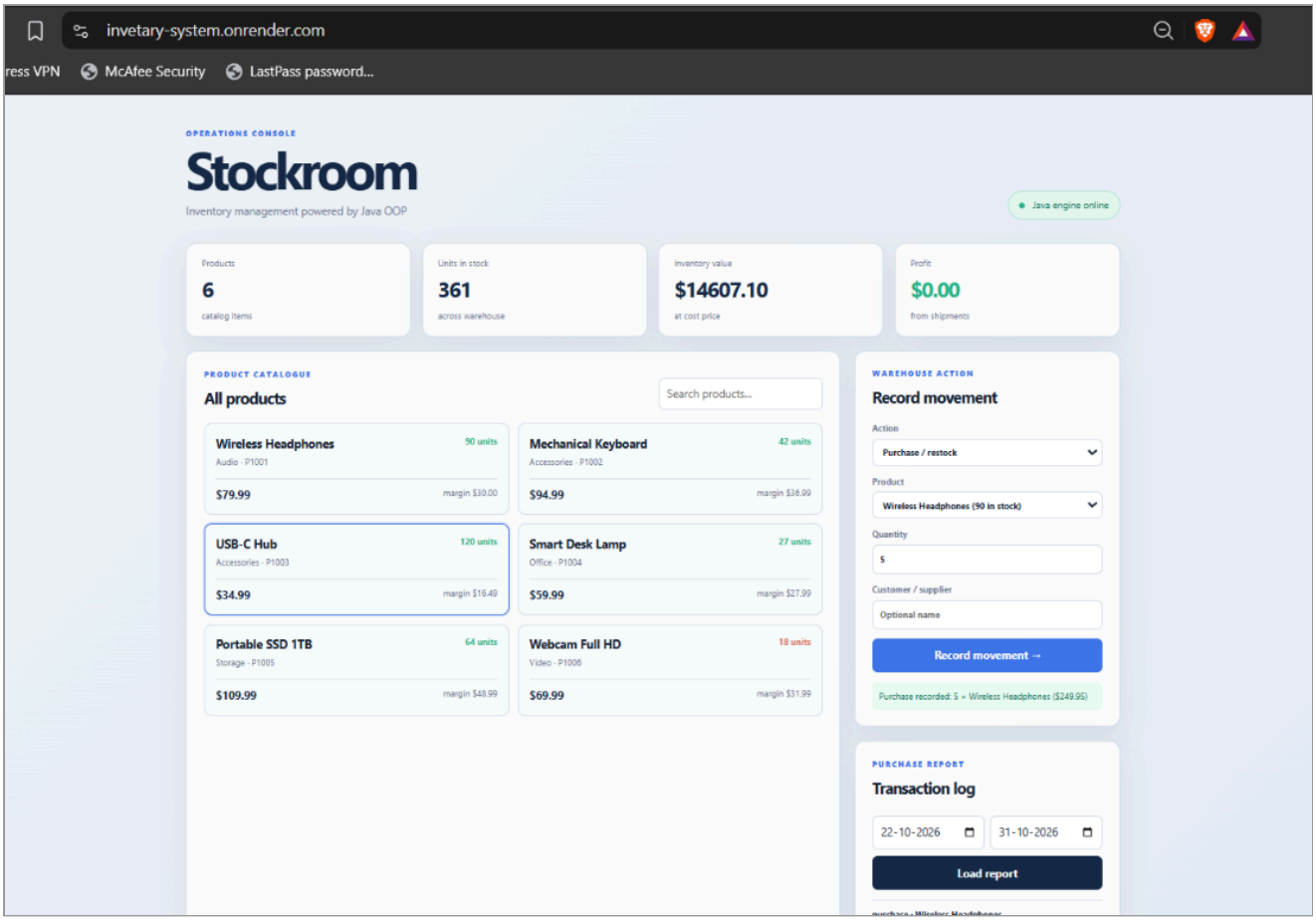


Figure 2: Stockroom after a purchase/restock operation with updated units and transaction report controls.

Observed Results

- The application reports six catalogue products.
- The initial dashboard shows 356 units in stock.
- Inventory value is calculated from product cost prices.
- Purchase operations update stock immediately.
- Shipping operations are blocked when stock is insufficient.
- The interface indicates Java engine availability.
- Transaction history can be filtered by dates.

10. CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

The Inventory Management System successfully implements the operations required in the mini project problem statement. It uses Java object-oriented design to model products, transactions and inventory responsibilities. The web interface makes the system easy to demonstrate, while the backend ensures that stock and financial calculations are performed consistently.

The project demonstrates how encapsulation, composition, collections, stream processing, validation and HTTP communication can be combined into a practical application. Docker deployment makes the Java project reproducible and suitable for hosting on Render.

10.2 Limitations

The current version stores data in memory. Therefore, data returns to the seeded state when the server restarts. The current version also uses a simple product list and does not include user authentication or a persistent database.

10.3 Future Scope

- Add a relational database for permanent product and transaction storage.
- Add product creation, editing and deletion.
- Add supplier and customer classes.
- Add user login and role-based permissions.
- Add invoices, discounts, tax and shipping cost fields.
- Add exportable PDF or CSV reports.
- Add dashboard charts and reorder notifications.
- Use a repository layer and indexed product lookup for larger datasets.

REFERENCES

1. Java Platform Standard Edition documentation, Java language and collections concepts.
2. Java HTTP Server API documentation.
3. HTML, CSS and JavaScript web platform documentation.
4. Docker documentation for multi-stage image builds.
5. Render documentation for web services, Docker deployment and health checks.
6. Project repository: `https://github.com/Akram2903/INVETARY_SYSTEM`